Lab2 Heterogeneous 3D IC Partitioning

Introduction

Partitioning in a 3D IC assigns cells to stacked dies and determines which nets need inter-die connections. In this lab, the dies use different standard-cell libraries, so each cell type has a separate area on each die.

Please implement a C++ partitioner that assigns every cell to the top or bottom die. Minimize the number of cross-die nets while satisfying both die area capacities and the cell-count balance constraint.

Problem definition

Model the circuit as a hypergraph $H = (V, E)$, where vertices are cell instances and hyperedges are nets. Partition V into disjoint sets T and B for the top and bottom dies; every cell must belong to exactly one set.

For each net e , let $c(e) = 1$ if it contains cells on both dies, and 0 otherwise. All nets have unit weight. Minimize the cut count:

$$N_{cut} = \sum_{e \in E} c(e)$$

Each net contributes at most one, regardless of its number of connected instances.

Let $a_T(v)$ and $a_B(v)$ be the areas of cell v on the two dies, and C_T and C_B their usable cell-area capacities. Both area constraints must hold:

$$\sum_{v \in T} a_T(v) \leq C_T; \quad \sum_{v \in B} a_B(v) \leq C_B$$

Define the balance ratio $r = \frac{|T|}{|T|+|B|}$, r must be between lower_bound and upper_bound.

Input specification

```
BALANCERATIO <upper_bound> <lower_bound>
LIBCELLS <L> //the number of cells in the library
LIBCELL <type_name> <area_top> <area_bottom> // the name and area info of each cell in the library
.....                    (L-1 more records)
DIECAP <capacity_top> <capacity_bottom> //area capacity
INSTS <N>  //the number of instances in the design
INST <instance_name> <type_name>
.....                    (N-1 more records)
NETS <M>
NET <net_name> <k> < instance_name1> ... < instance_namek>
.....                    (M-1 more records)
```

There is no comment in real input file.

Each type has two positive integer areas; all instances of that type use the same values.

Each net lists k distinct instances. An instance appears at most once per net, and every instance belongs to at least one net. Different nets with identical connectivity are counted separately.

The balance ratio must satisfy lower_bound ≤ r ≤ upper_bound.

Library types L	$1 \leq L \leq 10^3$
Cell instances N	$2 \leq N \leq 10^5$
Nets M	$1 \leq M \leq 2 * 10^5$
Net Degree k	$2 \leq k \leq N \leq 10^5$
Balance Ratio Upper/Lower Bound	$0 < bound < 1$ (floating point, round to 3 decimal places)
Area of one library cell	$1 \leq area \leq 10^5$
Capacity of one die	$1 \leq capacity \leq 10^9$
type/instance/net name	string with no spaces, unique within each category

Output specification

Write BALANCERATIO and r, round to 3 decimal places, CUTSIZE and the cut count, TOP and its cell count followed by its cell names, then BOTTOM as well.

The TOP and BOTTOM counts must be nonnegative integers, sum to N, and match the listed cells.

Every input cell must appear exactly once;

names within each die may be in any order.

Incorrect cut counts, unknown or repeated names, missing cells, extra tokens, or violations of area or balance constraints make the output invalid.

```
BALANCERATIO <$\frac{|T|}{|T|+|B|}$>
CUTSIZE <cut_count>
TOP <|T|>
< instance_name$_1$> ... < instance_name$_T$>
BOTTOM <|B|>
< instance_name$_1$> ... < instance_name$_B$>
```

Example input

```
BALANCERATIO 0.55 0.45
LIBCELLS 3
LIBCELL G1 10 6
LIBCELL G2 15 10
LIBCELL G3 20 11
DIECAP 45 30
INSTS 6
INST U1 G1
INST U2 G2
INST U3 G3
INST U4 G1
INST U5 G2
INST U6 G3
NETS 6
NET N1 2 U1 U2
NET N2 3 U1 U2 U3
NET N3 2 U3 U4
NET N4 2 U4 U5
NET N5 3 U4 U5 U6
NET N6 3 U2 U5 U6
```

Example output

```
BALANCERATIO 0.5
CUTSIZE 2
TOP 3
U1 U2 U3
BOTTOM 3
U4 U5 U6
```

Execution

Provide a Makefile that builds an executable named **Lab2**. The evaluator will compile and run:

```
make
./Lab2 <input_file> <output_file>
```

Before submission, compile and test your final program on the course server and verify its output.

A submission that fails to compile or run on the server receives zero marks.

For each case, the run time limit is up to 60 seconds. It will be regarded as "failed" if you use more than 60 seconds.

Hidden cases will be evaluated

A verifier is released to evaluate your result. `./verifier <input_file> <output_file> `

Submission

1. Please put all required files in a folder, the name of the folder is your Student_ID.

   Student_ID/

   Source code (.cpp, .h)

   Makefile

2. Use the below command to compress the folder in the linux environment, and the archived file name should be the same as your Student_ID.

   tar cvf Student_ID.tar Student_ID

For example, a student with ID 310510161 may construct the following folder:

310510161/

Lab2.cpp

Makefile

Create its archive with:

tar cvf 310510161.tar 310510161

3. Upload to e3. Upload Student_ID.tar. Submissions in any format other than .tar receive zero marks.

Evaluation

Component	Weight	Criterion
Validity	70%	Valid output satisfying all constraints
Quality	25%	Cut count of valid outputs; lower is better
Run_time	5%	Run time; lower is better